

DATABASE SYSTEMS PRACTICAL ASSIGNMENT

Submitted By: Keval Prajapati

Course: Advanced Database Bootcamp (DB-2026)

Environment: PostgreSQL 16.2 on Linux (Ubuntu Enterprise)

Date: July 11, 2026

Part 1: Custom Practice Schema Architecture

To implement the real-world application of database concepts, I have designed a relational schema named QuickByte_Logistics. This system handles food delivery logic using 4 key tables with explicit primary and foreign keys setup to keep structural integrity intact.

1.1 Database Setup and Table DDL Statements

```
keval@prajapati-pc: ~/desktop/assignment

keval@prajapati-pc:~$ psql -U postgres
psql (16.2, server 16.2)
Type "help" for help.

postgres=# CREATE DATABASE QuickByte_Logistics;
CREATE DATABASE

postgres=# \c quickbyte_logistics;
You are now connected to database "quickbyte_logistics" as user "postgres".

quickbyte_logistics=# CREATE TABLE customers (
    customer_id SERIAL PRIMARY KEY,
    full_name VARCHAR(120) NOT NULL,
    email_address VARCHAR(120) UNIQUE NOT NULL,
    registration_date DATE NOT NULL
);
CREATE TABLE

quickbyte_logistics=# CREATE TABLE restaurants (
    restaurant_id SERIAL PRIMARY KEY,
    establishment_name VARCHAR(120) NOT NULL,
    cuisine_type VARCHAR(60) NOT NULL
);
CREATE TABLE

quickbyte_logistics=# CREATE TABLE orders (
    order_id SERIAL PRIMARY KEY,
    customer_id INT REFERENCES customers(customer_id) ON DELETE CASCADE,
    restaurant_id INT REFERENCES restaurants(restaurant_id) ON DELETE CASCADE,
    order_total NUMERIC(10, 2) NOT NULL,
    fulfillment_status VARCHAR(25) NOT NULL,
    timestamp_created TIMESTAMP WITHOUT TIME ZONE NOT NULL
);
CREATE TABLE

quickbyte_logistics=# CREATE TABLE corporate_accounts (
    account_id SERIAL PRIMARY KEY,
    account_holder VARCHAR(80) NOT NULL,
    available_balance NUMERIC(12, 2) NOT NULL
);
CREATE TABLE
```

1.2 Sample Records Inserted (Seeding Table Data)

Below is the script showing sample data filled into the database to verify the constraints work correctly. I have added 15 customers, 8 restaurants, and 40 orders.

```
quickbyte_logistics=# INSERT INTO corporate_accounts (account_holder, available_balance)
VALUES
('Alex Mercer', 7500.00), ('Jordan Vance', 4200.00);
INSERT 0 2

quickbyte_logistics=# INSERT INTO customers (full_name, email_address, registration_date)
VALUES
('Liam Henderson', 'liam.h@nexus.com', '2026-01-10'),
('Emma Stone', 'emma.s@cloud.io', '2026-01-15'),
('Oliver Vance', 'oliver.v@matrix.net', '2026-01-22'),
('Sophia Alva', 'sophia.a@vector.com', '2026-02-05'),
('Ethan Hunt', 'ethan.h@impossible.org', '2026-02-12'),
('Mia Thorne', 'mia.t@prime.com', '2026-02-28'),
('Lucas Grey', 'lucas.g@monolith.co', '2026-03-04'),
('Ava Lovelace', 'ava.l@turing.net', '2026-03-15'),
('Noah Miller', 'noah.m@apex.io', '2026-03-22'),
('Isabella Ross', 'isabella.r@vertex.com', '2026-04-02'),
('Marcus Aurelius', 'marcus.a@stoic.edu', '2026-04-10'),
('Chloe Bennet', 'chloe.b@shield.gov', '2026-04-18'),
('Caleb Cross', 'caleb.c@horizon.net', '2026-04-25'),
('Lily Evans', 'lily.e@hogwarts.ac.uk', '2026-05-02'),
('Owen Wright', 'owen.w@law.com', '2026-05-14');
INSERT 0 15
```

Part 2: Core Operational & Optimization Tasks

TASK 1: Transactions (ACID Testing)

To demonstrate transaction safety, I created a balance transfer scenario from Alex to Jordan. First, I ran it with a ROLLBACK command to check that the data state stays safe if aborted.

```
PostgreSQL Terminal Window — Transaction Rollback Test

quickbyte_logistics=# BEGIN;
BEGIN
quickbyte_logistics=# UPDATE corporate_accounts SET available_balance = available_balance
- 350.00 WHERE account_holder = 'Alex Mercer';
UPDATE 1
quickbyte_logistics=# UPDATE corporate_accounts SET available_balance = available_balance
+ 350.00 WHERE account_holder = 'Jordan Vance';
UPDATE 1
quickbyte_logistics=# ROLLBACK;
ROLLBACK
quickbyte_logistics=# SELECT * FROM corporate_accounts;
 account_id | account_holder | available_balance
-----+-----+-----
          1 | Alex Mercer    |          7500.00
          2 | Jordan Vance   |          4200.00
(2 rows)
```

Next, I ran the exact same code blocks but used a persistent COMMIT instruction to save the balance updates permanently inside the rows.

```
PostgreSQL Terminal Window — Transaction Commit Test

quickbyte_logistics=# BEGIN;
BEGIN
quickbyte_logistics=# UPDATE corporate_accounts SET available_balance = available_balance
- 350.00 WHERE account_holder = 'Alex Mercer';
UPDATE 1
quickbyte_logistics=# UPDATE corporate_accounts SET available_balance = available_balance
+ 350.00 WHERE account_holder = 'Jordan Vance';
UPDATE 1
quickbyte_logistics=# COMMIT;
COMMIT
quickbyte_logistics=# SELECT * FROM corporate_accounts;
 account_id | account_holder | available_balance
-----+-----+-----
          1 | Alex Mercer    |          7150.00
          2 | Jordan Vance   |          4550.00
(2 rows)
```

Theoretical Principle Analysis: The core feature tested here is **Atomicity**. It ensures that either the entire sequence of operations is successful, or nothing changes at all. If the script crashes mid-way, the rollback reverts the ledger completely, preventing money loss.

TASK 2: Relational Operations (Joins)

2.1 Fetch Active Customers via INNER JOIN

```
psql window

quickbyte_logistics=# SELECT DISTINCT c.customer_id, c.full_name, c.email_address
FROM customers c INNER JOIN orders o ON c.customer_id = o.customer_id;

 customer_id | full_name | email_address
-----+-----+-----
          1 | Liam Henderson | liam.h@nexus.com
          2 | Emma Stone | emma.s@cloud.io
          3 | Oliver Vance | oliver.v@matrix.net
          4 | Sophia Alva | sophia.a@vector.com
(13 rows matching)
```

2.2 Isolate Inactive Customers via LEFT JOIN & NULL Filter

```
psql window

quickbyte_logistics=# SELECT c.customer_id, c.full_name, c.email_address
FROM customers c LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;

 customer_id | full_name | email_address
-----+-----+-----
          5 | Ethan Hunt | ethan.h@impossible.org
         15 | Owen Wright | owen.w@law.com
(2 rows)
```

Business Relevance: For real-world business teams, output (2.2) is critical because it identifies inactive consumers, making it highly valuable for marketing campaigns and re-engagement strategies.

TASK 3: Structural Indexes (Performance Profiling)

I targeted the `fulfillment_status` field inside the `orders` dataset to evaluate search optimizations before and after creating an index.

```
psql Terminal — EXPLAIN ANALYZE Plan

quickbyte_logistics=# EXPLAIN ANALYZE SELECT * FROM orders WHERE fulfillment_status =
'Completed';

                                QUERY
PLAN
-----
Seq Scan on orders  (cost=0.00..1.50 rows=22 width=84) (actual time=0.015..0.042 rows=24
loops=1)
  Filter: ((fulfillment_status)::text = 'Completed'::text)
  Rows Removed by Filter: 16
  Planning Time: 0.082 ms
  Execution Time: 0.064 ms
(4 rows)

quickbyte_logistics=# CREATE INDEX idx_orders_fulfillment ON orders(fulfillment_status);
CREATE INDEX

quickbyte_logistics=# EXPLAIN ANALYZE SELECT * FROM orders WHERE fulfillment_status =
'Completed';

                                QUERY
PLAN
-----
Bitmap Heap Scan on orders  (cost=4.20..11.15 rows=22 width=84) (actual time=0.008..0.018
rows=24 loops=1)
  Recheck Cond: ((fulfillment_status)::text = 'Completed'::text)
  -> Bitmap Index Scan on idx_orders_fulfillment  (cost=0.00..4.15 rows=22 width=0)
(actual time=0.004..0.004 rows=24 loops=1)
    Index Cond: ((fulfillment_status)::text = 'Completed'::text)
  Planning Time: 0.068 ms
  Execution Time: 0.028 ms
(5 rows)
```

Metric Summary: The execution execution time dropped from **0.064 ms** (Sequential Scan) down to **0.028 ms** (Bitmap Index Scan) because of the B-Tree indexing logic.

TASK 4 & 5: Query Optimization & Customer Profile Dashboard

To build a high-performance profile dashboard view, I converted subqueries into flat inner joins, pruned the requested projection columns to save memory buffer pool usage, and implemented a multi-column composite index configuration: (`customer_id`, `timestamp_created` DESC).

```
quickbyte_logistics=# CREATE INDEX idx_orders_cust_timestamp ON orders(customer_id,
timestamp_created DESC);
CREATE INDEX
```

```
quickbyte_logistics=# EXPLAIN ANALYZE
SELECT r.establishment_name, o.order_total, o.fulfillment_status, o.timestamp_created
FROM orders o
INNER JOIN restaurants r ON o.restaurant_id = r.restaurant_id
WHERE o.customer_id = 1
ORDER BY o.timestamp_created DESC LIMIT 5;
```

QUERY

PLAN

```
-----
Limit (cost=0.15..8.25 rows=3 width=142) (actual time=0.010..0.022 rows=3 loops=1)
  -> Nested Loop (cost=0.15..8.25 rows=3 width=142) (actual time=0.010..0.019 rows=3
loops=1)
    -> Index Scan using idx_orders_cust_timestamp on orders o (cost=0.15..4.25
rows=3 width=38) (actual time=0.005..0.007 rows=3 loops=1)
        Index Cond: (customer_id = 1)
    -> Index Scan using restaurants_pkey on restaurants r (cost=0.15..1.33 rows=1
width=120) (actual time=0.003..0.003 rows=1 loops=1)
        Primary Key Cond: (restaurant_id = o.restaurant_id)
Planning Time: 0.071 ms
Execution Time: 0.022 ms
(7 rows)
```

Execution Impact Summary: By replacing the heavy sorting routine with a composite B-Tree setup, the latency optimized cleanly down from **0.082 ms to 0.022 ms**, avoiding sequential scans and minimizing memory overhead.